Clase 1 · Imagen de un proceso, segmentos de memoria y árbol de procesos

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 25 ago 2026 · Tema 1 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Encuadre y entorno

- La primera parte es "una continuación de sistemas operativos": procesos y luego servidores. **Uniprocesador** (sin multinúcleo). "Distribuido" = servidor y cliente en hosts distintos por red.
- Código en un repositorio público de GitHub; apuntes en GitHub Pages (el sitio viejo HTTP está obsoleto).
- **Codespaces:** contenedor con root; si lo rompes (`rm -rf`) lo destruyes y creas otro. También Linux propio, WSL o SSH.
- IPs públicas de la UNAM: **132.247.x.x y 132.248.x.x**; las 192.168.x.x son privadas.
- Documentación de funciones C: **The Open Group**.

2. Imagen de un proceso

"Una fotografía o instantánea": el ejecutable en ejecución crea un proceso que vive mientras se ejecuta. **Demonios** (daemons) corren en segundo plano; cerrar la ventana no siempre mata el proceso.

Segmento	Tipo	Contenido
text	Estático	Código máquina y strings de `printf` (solo lectura)
data	Estático	Globales **inicializadas**
BSS	Estático	Globales **no inicializadas**
heap	Dinámico	`malloc`
stack	Dinámico	Locales y parámetros

Heap y stack crecen en sentidos opuestos. Scope de una local = su función.

> **PCB (examen)**
>
> **BCP / PCB** (Process Control Block): estructura del kernel con los metadatos del proceso. El profesor lo marcó como muy importante.
>
> Error común: creer que las globales no inicializadas van en data (van en BSS).

3. Ejercicio: servidor en C con dos terminales

- Compilar con `-o` y el mismo nombre que el `.c` (sin `-o` sale `a.out`, mala práctica).

- Terminal 1: el servidor con `getchar()` para que no muera; Terminal 2: inspeccionarlo.

4. size, pmap y /proc

- **`size servidor`** (estático, archivo en disco): text, data, bss, dec, hex. Ej.: 1767 + 592 + 8 = **2367** (0x93F).

> **¿Por qué size no muestra heap ni stack? (examen)**
>
> Porque son **dinámicos**: solo existen en RAM con el proceso vivo; `size` analiza el archivo estático.
>
> - **`pmap <PID>`** (dinámico, desde el kernel): stack, heap, librerías `.so`. Valores en **páginas de 4 KB** (múltiplos de 4). Stack y heap tienen R/RW pero **nunca X**, para impedir inyectar código.
> - **`cat /proc/<PID>/maps`**: direcciones, permisos y librerías mientras el proceso vive.

5. Árbol de procesos

- Raíz **PID 1**: init (clásico) o **systemd** (moderno).
- **PID** propio y **PPID** del padre. Todo comando lanzado en una terminal tiene como padre a su **bash** (`ps -e -o pid,ppid`). El PID de bash no cambia; el de `ps` sí.

6. top, ps y ls

- **top:** dinámico (q para salir). **Tiempo compartido:** el planificador reparte lapsos de CPU por prioridad → ilusión de paralelismo.
- **ps:** estático; sin opciones, solo la terminal actual. **`ps aux`:** USER, PID, %CPU, %MEM, VSZ, RSS, TTY, STAT, START, TIME, COMMAND. Demonios: `?` en TTY y nombre terminado en **d** (systemd, kthreadd).

> **Sintaxis (examen)**
>
> En `ps -e -o pid,ppid,user,stat,cmd` las columnas van **separadas por comas y sin espacios**; un espacio da error.
>
> - **`ls -ltr`:** largo, por fecha, invertido: lo más reciente queda junto al prompt (útil en incidentes de seguridad).
> - TTY viene de los **teletipos** de los años 40–50.

7. Race condition y deadlock

- Procesos compiten por la CPU → **condición de carrera**.
- **Deadlock:** "el proceso 1 necesita que termine el 2, y el 2 que termine el 1". No diseñar dependencias circulares.

Pendiente

Sin entrega; repasar el sitio y el PDF 2 del Tema 1. Próxima: `fork()`.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
Segmentos	text, data (inicializadas), BSS (no inicializadas), heap (malloc), stack (locales)
size vs. pmap	Estático en disco / dinámico en RAM (páginas de 4 KB)
PCB	Metadatos del proceso en el kernel
PID 1	init / systemd
PPID de un comando	El PID de su bash
ps -o	Columnas con comas y sin espacios
Deadlock	Dependencia circular entre procesos